Name: _____ Student NUID: _____

CS 2100: Program Design and Implementation 1

Practice Quiz 3

Instructions

- Do not begin the quiz until instructed to do so.
- You may use both sides of a sheet of paper up to 8.5"x11" for reference, but no other resources, including phones, computers, AI, headphones, and ear pods.
- You have until the end of the class period to complete the quiz.
- Students may not leave the classroom during the first 10 minutes of the quiz (except in case of emergency).
- Hand your completed quiz to an instructor before leaving the room.
- Talk to an instructor if you need to leave the room and reenter.

1. Classes (revisited from Quiz 2)

Please read the `Triangle` class below.

Add three methods to the `Triangle` class:

1. `is_equilateral`, which returns `True` if all three sides of the `Triangle` are equal (and `False` otherwise) 2. `is_isosceles`, which returns `True` if any two (or all three) sides of the `Triangle` are equal (and `False` otherwise) 3. `is_scalene`, which returns `True` if all three sides of the triangle are of different sizes (and `False` otherwise)

Documentation is not graded for this problem.

```
class Triangle:
    """A triangle class that represents a triangle with three sides."""
    def __init__(self, a: float = 1, b: float = 1, c: float = 1) -> None:
        """Initialize the triangle with three sides.

        Args:
            a (float): The length of side a.
            b (float): The length of side b.
            c (float): The length of side c.

        Raises:
            ValueError: If the sides do not form a valid triangle.
        """
        if (a + b <= c) or (a + c <= b) or (b + c <= a) or (a <= 0) or (b <= 0) or (c <= 0):
            raise ValueError(f"Invalid triangle sides: {a}, {b}, {c}")
        self.a = a
        self.b = b
        self.c = c

    def __eq__(self, other: object) -> bool:
        """Check if two triangles are equal based on their sides. If the sides are the same
        lengths, even in a different order, the triangles are considered equal."""
        if not isinstance(other, Triangle):
            return False
        return sorted([self.a, self.b, self.c]) == sorted([other.a, other.b, other.c])
```

Solution:

```
def is_equilateral(self) -> bool:
    """Check if the triangle is equilateral (all sides are equal)."""
    return self.a == self.b == self.c

def is_isosceles(self) -> bool:
    """Check if the triangle is isosceles (at least two sides are equal)."""
    return self.a == self.b or self.a == self.c or self.b == self.c

def is_scalene(self) -> bool:
    """Check if the triangle is scalene (all sides are different)."""
    return self.a != self.b and self.a != self.c and self.b != self.c
```

2. Unit testing (revisited from Quiz 1)

Please read the `Triangle` class above.

Write at least four unit tests for the constructor in the `Triangle` class. Make sure to include at least one test for a `ValueError` and at least one test that a default value for an argument is used. Documentation is not graded for this problem.

```
import pytest

class TestTriangle:
    """Test suite for the Triangle class."""
```

Solution:

```
def test_invalid_triangle_all_positive(self) -> None:
    """Test creating an invalid triangle with all positive sides."""
    try:
        Triangle(1, 2, 4)
    except ValueError:
        assert True
    else:
        assert False

def test_invalid_triangle_with_negative_side(self) -> None:
    """Test creating an invalid triangle with a negative side."""
    try:
        Triangle(-1, 2, 3)
    except ValueError:
        assert True
    else:
        assert False

def test_invalid_triangle_with_zero_side(self) -> None:
    """Test creating an invalid triangle with a zero side."""
    try:
        Triangle(0, 2, 3)
    except ValueError:
        assert True
    else:
        assert False

def test_triangle_constructor_uses_default_values(self) -> None:
    """Test that the triangle constructor uses default values correctly."""
    triangle = Triangle()
    assert triangle.a == 1
    assert triangle.b == 1
    assert triangle.c == 1
```

3. Aliasing and equality (revisited from Quiz 2)

Please read the `Square` class below.

```
class Square:
    """A class representing a square."""
    def __init__(self, side: float) -> None:
        """Initialize a square with the given side length.

        Args:
            side : float
                The length of the side of the square.
        """
        self.side = side

    def __eq__(self, other: object) -> bool:
        """Check if two squares are equal based on their side length.

        Args:
            other (object): The other square to compare with.

        Returns:
            bool: True if the squares have the same side length, False otherwise.
        """
        if not isinstance(other, Square):
            return False
        return self.side == other.side
```

Then, please consider this code:

```
square1 = Square(1.0)
```

Which of these `Square`s is equal to `square1`? (Please circle all that are equal to `square1`, including aliases.)

```
square2 = square1

square3 = Square(1.0)

square4 = Square(square1.side)

square5 = Square(500.0)
```

Solution:

```
square2 = square1

square3 = Square(1.0)

square4 = Square(square1.side)
```

4. Mean and variance

4A: A list of 11 numbers has a mean of 30 and a variance of 20. If we remove one of the values, which equals 30, what are the new mean and variance?

4B: Consider the scenario where you are across town, and you need to take the bus or bike to a class that starts in 20 minutes. Suppose that biking there takes on average 18 minutes, and taking the bus takes on average 14 minutes.

A student says, "Taking the bus has a mean 4 minutes shorter, so it must be the better choice for arriving on time." Give a scenario where the bus is the right choice, and one where the bus is the wrong choice.

Solution:

```
a.
Mean: ((30 * 11) - 30) / 10 = 30
Variance: ((20 * 11) - 0^2) / 10 = 22

b.
Wrong choice: taking the bus has a large standard deviation (e.g., 10 minutes), and biking has a
small standard deviation (e.g., 1 minute), so biking is more reliably going to get you there on time.
Right choice: taking the bus has a small standard deviation (e.g., 2 minutes), and biking has an
equal or larger standard deviation (e.g., also 2 minutes), so the chance of getting there by bus is
still reliably faster.
```

5. Lists, sets, and dictionaries

Write a function called `invert` that takes a dictionary (where the keys and values are both words) and returns a new dictionary where each value of the original becomes a key in the new dictionary, and the value associated with that key is the list of all original keys that mapped to it. Documentation is not graded for this question.

Example usage:

```
invert({"apple": "red", "cherry": "red", "banana": "yellow"})
# {"red": ["apple", "cherry"], "yellow": ["banana"]}
```

Solution:

```
def invert(d: dict[str, str]) -> dict[str, list[str]]:
    """Invert a dictionary mapping strings to strings, producing a dictionary mapping strings to
lists of strings.

    Args:
        d (dict[str, str]): The dictionary to invert.

    Returns:
        dict[str, list[str]]: The inverted dictionary.
    """
    inverted: dict[str, list[str]] = {}
    for key, value in d.items():
        if value not in inverted:
            inverted[value] = []
        inverted[value].append(key)
    return inverted
```